



pymoolab

User Manual & Technical Reference

An Open-Source Visual Analytics Framework for Multi-Objective Optimization
using LLM-Based Code Generation and MCDM

METISBr — Brazilian Research Group on Multi-Objective and Many-Objective Optimization (MaOPs)

UFOP — Federal University of Ouro Preto, Brazil

Package version: 1.0.2 | Manual edition: 2026.07 | Generated on 2026-07-01

Contact: santostf+metisbr@ufop.edu.br

Contents

1. Introduction
2. Requirements and Installation
3. Project Architecture
4. Directory Structure
5. Core Modules (core/)
6. Plugin System
7. Implemented Catalog
8. The Graphical Interface (five tabs)
9. Reproducibility and Execution Manifests
10. Statistical Analysis and MCDM
11. Creating New Problems
12. Creating New Algorithms
13. Creating New Metrics
14. Creating New Operators
15. Execution and Workflow
16. Troubleshooting
- Appendix A. Array Backend Acceleration (CPU / JAX / MLX)
- Appendix B. References

1. Introduction

PymooLab is an open-source **visual analytics** framework for multi-objective and many-objective optimization, built on top of the **pymoo** ecosystem and the **PySide6** (Qt for Python) graphical interface. Its purpose is to give researchers a single environment in which to configure, run, analyze, and reproduce experimental benchmarks without writing any orchestration code of their own.

The entire graphical application lives in `PymooLab.py` (17,061 lines), whereas the reusable scientific logic is isolated in the `core/` package. The main window exposes five workspaces: **Quick Test**, **Experiment**, **Results**, **AI Agent**, and **Extensibility**.

1.1. Key Features

- Benchmark orchestration with parallel execution across isolated subprocesses (`ProcessPoolExecutor` plus a `QThread` bridge).
- Automatic discovery of local plugins — algorithms, problems, metrics, and operators — with no manual registration required.
- Built-in statistical analysis: Wilcoxon (pairwise against a reference) and Friedman (global ranking with Kendall's coefficient) tests, with CSV and LaTeX-table export.
- Interactive visualization of Pareto fronts and convergence curves.
- Optional **JAX** (GPU/TPU) and native **MLX** (Apple Silicon) acceleration for compatible problems, operators, and metrics, with device-aware backend selection (MLX is the default when detected).
- LLM-assisted formulation (**Anthropic**) of optimization artifacts (problems and metrics), with security validation of the generated code.
- Multi-criteria decision making (**MCDM**) for selecting a compromise solution from the obtained fronts (TOPSIS and weighted sum).
- Guaranteed reproducibility through seed plans and SHA-256-signed execution manifests.

PymooLab is developed by the **METISBr** group at UFOP (Federal University of Ouro Preto), with a focus on academic research in Multi-Objective and Many-Objective Optimization (MaOPs).

2. Requirements and Installation

2.1. Recommended Environment

Item	Value
Python	3.11.x
Default backend	CPU (NumPy)
Optional backend	JAX (GPU/TPU)
Apple Silicon backend	MLX (GPU/NPU)
GUI	PySide6 + qt-material
Tested systems	Windows 10/11, Linux, macOS

2.2. Base Dependencies

Package	Version	Role
numpy	>=2.3, <3	Linear algebra and numerical arrays
pymoo	>=0.6.1, <0.7	Multi-objective optimization core
scipy	>=1.16, <2	Statistical tests (Wilcoxon/Friedman)
matplotlib	>=3.10, <4	Convergence and Pareto plots
psutil	>=7.2, <8	Process monitoring
PySide6	>=6.10, <7	Qt graphical interface
qt-material	>=2.17, <3	Application visual theme
qt-material-icons	>=0.4, <1	Navigation iconography

2.3. Optional Profiles

Profile	Packages	Use
GPU/JAX	jax, jaxlib	GPU/TPU acceleration
CuPy (legacy)	cupy-cuda12x	Benchmark helpers / GPU experiments
LLM	anthropic	AI-assisted code generation
ML	scikit-learn	ML-dependent algorithms/problems
Dev	pytest	Automated tests

2.4. Installation

Manual installation of the base stack and the optional profiles:

```

pip install numpy pymoo scipy PySide6 qt-material qt-material-icons matplotlib psutil
pip install jax jaxlib                # optional: GPU/JAX
pip install anthropic                 # optional: LLM agent
pip install scikit-learn              # optional: ML algorithms
pip install pytest                    # optional: development

```

On Windows, the `install_windows.bat` script detects the Python 3.11 interpreter and installs the base dependencies automatically. To launch the application:

```
python PymooLab.py
```

3. Project Architecture

PymooLab follows a layered, modular architecture. The presentation layer (GUI) is kept strictly separate from the scientific logic, ensuring that interface adjustments never alter execution behavior, result contracts, or the core APIs.

Layer	Module(s)	Responsibility
GUI	PymooLab.py, styles.py	PySide6 graphical interface and centralized styling
Core logic	core/	Statistical analysis, execution, LLM, MCDM, and registry
Base classes	core/	pymoo compatibility wrappers (backend-aware)
Plugins	algorithms/, problems/, metrics/, operators/	Implementations auto-discovered at startup
Utilities	util/	Array backend, display, NDS, dominance, reference directions
Worker	pymoolab_process_worker.py	Trial execution in isolated subprocesses

3.1. PymooLab.py — The Main Application

The main file concentrates the entire GUI application. Its central elements are:

- **PymooExperimentWindow** — the main window (titled “PymooLab - A Graphical Framework for pymoo”) that hosts the five tabs and coordinates the execution engine.
- **Plugin discovery** — automatic scanning of the plugin directories, producing the typed specs for each domain.
- **Execution engine** — parallelizes trials through `ProcessPoolExecutor`, bridged to the UI by a `QThread` so the interface stays responsive.
- **Analysis workspace** — statistical tests, convergence curves, and Pareto fronts, together with the CSV / LaTeX export system.

3.2. core/ — Modular Logic

Subpackage	Module	Function
analysis	stat_tests.py	Wilcoxon, Friedman, and statistical summaries
execution	backend_runtime.py	JAX/GPU detection and device status
execution	reproducibility.py	Seed plans and SHA-256 manifests
execution	qt_bridges.py	QObject bridge for asynchronous LLM tasks
llm	formulation.py	LLM artifact generation and validation service
mcdm	decision.py	Compromise-solution selection (TOPSIS / sum)
registry	backends.py	Backend descriptor registry (CPU / JAX / MLX tokens)
registry	backend_selection.py	Backend-aware resolution of specs/operators
registry	plugin_dirs.py	Scaffolding of the plugin directories
registry	rollout.py	Feature gating by rollout stage
registry	specs.py	AlgorithmSpec/ProblemSpec/... dataclasses

4. Directory Structure

```
pymoolab/
|-- PymooLab.py           # Main GUI application (17,061 lines)
|-- styles.py            # Centralized design system (AppStyles)
|-- pymoo_metadata.py     # pymoo algorithm/problem metadata
|-- pymoolab_process_worker.py # Subprocess worker (run_trial_in_process)
|-- optimize.py          # Alias for pymoo.optimize (from optimize import minimize)
|-- version.py / __init__.py # Package version and metadata
|-- requirements.txt      # Dependency manifest
|-- install_windows.bat  # Windows dependency installer
|-- app.png / pymoolab.png # Application images
|
|-- core/                 # Unified core: base classes + modular logic
|   |-- algorithm.py      # Algorithm base (GPU/backend-aware)
|   |-- problem.py / population.py / individual.py
|   |-- mating.py / parameters.py / duplicate.py
|   |-- analysis/stat_tests.py
|   |-- execution/backend_runtime.py, reproducibility.py, qt_bridges.py
|   |-- llm/formulation.py
|   |-- mcdm/decision.py
|   |-- registry/specs.py, plugin_dirs.py, backend_selection.py, rollout.py, backends.py
|
|-- algorithms/           # 293 local algorithms (one subfolder each)
|-- problems/             # Optimization problems
|   |-- single/ (BBOB, CEC2008/2013/2020, ...)
|   |-- multi/ (ZDT, CF, UF, MW, DASCOP, RWMOP, ...)
|   |-- many/ (DTLZ, WFG, MAF, ZCAT, GLSMOP, LSMOP, MAOPP)
|-- metrics/              # Quality metrics (community_metrics + JAX)
|-- operators/            # crossover/, mutation/, selection/, sampling/,
|                          # repair/, survival/, utility_functions/
|-- util/                 # array_backend, nds/, display/, dominator, ref_dirs
|-- tests/                # Benchmarks and tests (excluded from releases)
```

Note. Missing plugin directories are created automatically at startup by `ensure_plugin_directories()`. Files such as `last_experiment_config.json`, `pymoo_config.json`, and the `mcdm_results/` folder are runtime-generated artifacts.

5. Core Modules (core/)

5.1. core/algorithm.py

The `Algorithm` class extends `pymoo.core.algorithm.Algorithm` with transparent GPU/JAX backend support. Local algorithms should inherit from it to benefit from backend selection.

```
from core.algorithm import Algorithm

class MyAlgorithm(Algorithm):
    def __init__(self, *args,
                 use_gpu: bool = False,
                 array_backend: str = "auto", # 'auto' | 'numpy' | 'jax' | 'mlx'
                 gpu_dtype: str = "float32",
                 **kwargs):
        self.use_gpu_requested = bool(use_gpu)
        self.array_backend_requested = array_backend
        super().__init__(*args, **kwargs)

    def get_array_module(self):
        return array_backend.xp # numpy, jax.numpy, or mlx.core, per the active backend
```

5.2. core/registry/specs.py — Registry Dataclasses

Every discovered plugin is represented by a typed dataclass that encapsulates its metadata and its instantiation *factory*.

Dataclass	Key fields	Purpose
AlgorithmSpec	id, name, source, module, factory, flags	Algorithm record with scope flags (single/multi/many/neural)
ProblemSpec	id, name, source, module, factory, default_n_var, default_n_obj	Problem record with default dimensions (30 var, 2 obj)
MetricSpec	id, name, source, module, factory	Quality-metric record
OperatorSpec	id, name, source, category, module, class_name	Genetic-operator record, by category

All of them expose a `label` property in the `"{name} [{source}]"` format, used in the GUI lists. `AlgorithmSpec.is_factory_callable` distinguishes `create_algorithm` wrappers from direct classes.

5.3. Core Services

- **backend_runtime.detect_gpu_runtime()** — inspects JAX, counts devices, and identifies GPU/CUDA; **build_gpu_status_text()** produces the message shown in the UI.
- **reproducibility.build_seed_plan(config, total_slots)** — produces the seed plan; **build_execution_manifest(...)** generates the canonical manifest with a SHA-256 hash.
- **backend_selection.select_specs_for_backend(...)** and **resolve_available_operator_target(...)** — choose the NumPy, JAX, or MLX variant of each artifact at runtime.
- **rollout.rollout_stage()** / **rollout_allows_domain(domain)** — control which domains take part in backend-aware loading.

6. Plugin System

PymooLab uses directory-based automatic discovery. At startup, the application ensures the plugin folders exist and scans each one, registering valid modules as *specs*.

6.1. Discovery Flow

```
Startup
-> ensure_plugin_directories(base_dir) # creates algorithms/ problems/ metrics/ operators/
-> Scan algorithms/ : subfolder + __init__.py -> AlgorithmSpec
-> Scan problems/   : .py file per family      -> ProblemSpec
-> Scan metrics/    : community_metrics(.py)    -> MetricSpec
-> Scan operators/  : subfolder per category    -> OperatorSpec
-> Scan pymoo.*     : native algorithms/problems/operators
-> select_specs_for_backend(...) # resolves NumPy / JAX / MLX variants
```

6.2. Registration Contracts

Domain	Required export	Registration
Algorithm	Class inheriting Algorithm or create_algorithm(config)	AlgorithmSpec with factory
Problem	Class inheriting pymoo.core.problem.Problem	ProblemSpec with factory
Metric	Callable (front, context) -> float, in METRICS	MetricSpec with factory
Operator	Class inheriting pymoo.core.{crossover, mutation, ...}	OperatorSpec with module + class_name

6.3. ALGORITHM_FLAGS

Each algorithm module may declare an ALGORITHM_FLAGS dictionary indicating the problem types it supports. The accepted values are **single**, **multi**, **many**, and **neural**. These flags drive the GUI filters.

```
ALGORITHM_FLAGS = {
    "MyAlgorithm": {"multi", "many"},
}
```

7. Implemented Catalog

Snapshot verified against the codebase on 2026-07-01. The catalog combines local plugins with native pymoo artifacts.

7.1. Quantitative Summary

Domain	Local count	Note
Algorithms	293 subfolders	One directory per algorithm under algorithms/
Problem families	47 modules	7 single + 33 multi + 7 many
Metrics	25 functions	community_metrics.py (+ JAX variant and Monte Carlo HV)
Operators	68 modules	crossover 24, mutation 12, repair 12, utility 12, selection 4, sampling 4

7.2. Problem Families

Single-objective (problems/single): BBOB, CEC2008, CEC2013, CEC2020, MULTITASKING_SOPS, REALWORLD_SOPS, SIMPLE_SOPS.

Multi-objective (problems/multi): BT, CF, DASCOP, DOC, DSMOP, FCP, FDA, IMMOEA, IMOP, LIRCMOP, LRMOP, LSCM, LSMMOP, MMF, MMMOP, MOEADDE, MOEADM2M, MULTITASKING_MOPS, MW, REALWORLD_MOPS, RMMEDA, RWMOP, SDC, SMD, SMMOP, SMOP, SPARSE_REALWORLD_MOPS, TP, UF, VNT, ZDT, ZXH_CF.

Many-objective (problems/many): DTLZ, GLSMOP, LSMOP, MAF, MAOPP, WFG, ZCAT.

7.3. Quality Metrics

HV (Hypervolume), IGD, IGD+, GD, GD+, DeltaP, Spacing, Spread, IGD_X, PD, CPF, DM, Feasible_rate, and Min_value, along with aggregates for multitask benchmarks (Mean_HV, Mean_IGD, Worst_HV, Worst_IGD, Task1/Task2_HV/IGD). The community_metrics_JAX.py variant provides accelerated implementations, and hv_fast_mc.py offers a Monte Carlo hypervolume estimate for high objective counts.

7.4. Operators

Crossover (24): SBX, PCX, Point/Two-Point/Single-Point, Uniform (UX), Order (OX), Exponential, Differential, among others. **Mutation (12):** Polynomial (PM), Bitflip, Inversion, Gaussian, etc. **Selection (4):** Tournament, Random, and others. **Sampling (4):** Latin Hypercube (LHS), Random, Permutation. Additional categories: **repair (12)**, **survival**, and **utility_functions (12)**.

8. The Graphical Interface (five tabs)

The main window organizes the workflow into five tabs, accessible from the top bar. The application starts maximized, with a qt-material theme and the plugin directories already created.

8.1. Quick Test (science icon)

Rapid configuration and execution of a single scenario. It lets you choose one algorithm, one problem, and a set of metrics, adjust the essential parameters (number of evaluations/generations, population size), and trigger an isolated run with immediate visualization of the front and the convergence. The configuration can be saved to and loaded from `pymoo_config.json`.

8.2. Experiment (biotech icon)

A full benchmark orchestrator. It supports multiple selection (*checkboxes*) of several algorithms, problems, and metrics, along with the number of runs, the seed mode, and the execution backend. Experiments are parallelized across subprocesses and can be persisted as `pymoo_experiment_config.json` or reloaded from `last_experiment_config.json`. Filters by objective type (single/multi/many) and by family streamline the setup.

8.3. Results (analytics icon)

The analysis workspace for the collected results: tables aggregated by algorithm/problem/metric, convergence curves, Pareto fronts, statistical tests (Wilcoxon and Friedman), and export to CSV and LaTeX tables. It also hosts the MCDM panel for compromise-solution selection.

8.4. AI Agent (smart_toy icon)

The interface to the LLM-assisted formulation service (Anthropic). From a natural-language description, the agent generates code for **problems** and **metrics**, validates it (security analysis and contract conformance), and saves it as an artifact ready for discovery. It includes API-key management (persisted locally), automatic Anthropic model selection, and access validation.

Security. All generated code passes through `validate_problem_code` / `validate_metric_code` and through detailed validation of the *artifact bundle* before being persisted, preventing unsafe imports or constructs.

8.5. Extensibility (code icon)

Centralizes the manual creation of plugins and the inspection of the catalog. It offers templates and shortcuts for adding algorithms, problems, metrics, and operators following the contracts described in Chapters 11 through 14.

9. Reproducibility and Execution Manifests

Reproducibility is handled by the `core/execution/reproducibility.py` module, which builds deterministic seed plans and signed manifests.

9.1. Seed Modes

Mode	Behavior	Deterministic?
random	Independent random seed per trial	No
fixed	Same base seed for every trial	Yes
sequence	Explicit sequence (or base + step) of seeds	Yes

`build_seed_plan(config, total_slots)` returns a dictionary with `mode`, `deterministic`, `base_seed`, `step`, `provided_sequence`, and the final list of seeds. In *sequence* mode without an explicit sequence, seeds are generated as `base + idx * step`.

9.2. Execution Manifest

`build_execution_manifest(...)` records the sanitized configuration, the selected problems/algorithms/metrics, the seed plan, the execution backend, and an ISO *timestamp*. The content is serialized canonically and assigned a `manifest_sha256` field, making it possible to verify the integrity and provenance of every batch of experiments.

10. Statistical Analysis and MCDM

10.1. Statistical Tests

The `analysis/stat_tests.py` module offers two main procedures, both taking an `alpha` parameter (default 0.05) and a `higher_better` flag to indicate the metric's direction of improvement:

- **`run_wilcoxon(values_algo, values_ref, ...)`** — the Wilcoxon signed-rank test, pairwise against a reference algorithm; it reports the mean, standard deviation, p-value, effect size, and decision. It requires `min_samples` samples (default 5).
- **`run_friedman(blocks, ...)`** — the Friedman test for the global comparison of multiple algorithms; it reports the p-value, Kendall's coefficient of concordance (*W*), and the mean ranks per algorithm.

`summarize_stat_results(...)` consolidates the results into a structure ready for export (CSV / LaTeX), with the method label, the metric, the backend, and the global p-values.

10.2. MCDM — Compromise-Solution Selection

`mcdm/decision.py` implements `select_compromise_solution(front, method, weights_text)`. Objectives are normalized to `[0, 1]`; weights may be supplied (they must sum to a positive value and provide exactly one value per objective) or assumed uniform. Available methods:

Method	Criterion	Selection
topsis	Distance to the weighted ideal and anti-ideal solutions	Highest relative-closeness score
weighted-sum	Weighted sum of the normalized objectives	Lowest aggregate sum

The return value includes `index`, `score`, the selected vector, the normalized weights, and the method. The latest decision is persisted to `mcdm_results/last_mcdm_decision.json`.

11. Creating New Problems

11.1. File Location

Directory	Problem type	Examples
problems/single/	Single-objective (1 obj)	BBOB, CEC2008/2013/2020
problems/multi/	Multi-objective (2-3 obj)	ZDT, CF, UF, MW
problems/many/	Many-objective (≥ 4 obj)	DTLZ, WFG, MAF, ZCAT

11.2. Implementation Pattern

Every problem inherits from `pymoo.core.problem.Problem` and implements `_evaluate()`:

```
from pymoo.core.problem import Problem
import numpy as np

class MyProblem(Problem):
    def __init__(self, n_var: int = 10, **kwargs):
        super().__init__(
            n_var=int(n_var),
            n_obj=2,
            n_constr=0,
            xl=np.zeros(n_var),
            xu=np.ones(n_var),
        )

    def _evaluate(self, X, out, *args, **kwargs):
        f1 = X[:, 0]
        g = 1.0 + 9.0 * np.mean(X[:, 1:], axis=1)
        f2 = g * (1.0 - np.sqrt(f1 / g))
        out["F"] = np.column_stack([f1, f2])
```

The file is discovered automatically and registered as a `ProblemSpec` with the default dimensions defined in the constructor.

12. Creating New Algorithms

12.1. Directory Layout

```
algorithms/
|-- my_algorithm/
    |-- __init__.py      # Exports the class or the factory
    |-- my_algorithm.py  # Implementation + ALGORITHM_FLAGS
```

12.2. Implementation

```
# algorithms/my_algorithm/my_algorithm.py
from __future__ import annotations
import numpy as np
from core.algorithm import Algorithm
from core.population import Population

ALGORITHM_FLAGS = {"MyAlgorithm": {"multi", "many"}}

class MyAlgorithm(Algorithm):
    def __init__(self, pop_size: int = 100, **kwargs):
        super().__init__(**kwargs)
        self.pop_size = pop_size

    def _setup(self, problem, **kwargs):
        self.n_var, self.n_obj = problem.n_var, problem.n_obj
        self.xl, self.xu = problem.xl, problem.xu

    def _initialize_infill(self):
        X = np.random.uniform(self.xl, self.xu, (self.pop_size, self.n_var))
        return Population.new("X", X)

    def _infill(self):
        X = np.random.uniform(self.xl, self.xu, (self.pop_size, self.n_var))
        return Population.new("X", X)

    def _advance(self, infills=None, **kwargs):
        self.pop = infills
```

12.3. Export and Factory

```
# algorithms/my_algorithm/__init__.py
from .my_algorithm import MyAlgorithm
__all__ = ["MyAlgorithm"]

# Alternative: explicit factory recognized by the registry
def create_algorithm(config: dict):
    return MyAlgorithm(pop_size=config.get("pop_size", 100))
```

12.4. Reusing pymoo Building Blocks (Recommended)

Before writing an algorithm from scratch, reuse the building blocks that pymoo and core/ already provide. The vast majority of PymooLab's local algorithms simply **subclass an existing pymoo algorithm** (most often NSGA2) and override only the part that is genuinely new. Genetic operators — SBX crossover, polynomial mutation, tournament selection, sampling, and survival — are battle-tested; pass them in rather than reimplementing them.

```
# Subclass NSGA2 and reuse its operators end to end
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.operators.crossover.sbx import SBX
from pymoo.operators.mutation.pm import PM
from pymoo.operators.sampling.lhs import LHS

ALGORITHM_FLAGS = {"MyAlgorithm": {"multi", "many"}}

class MyAlgorithm(NSGA2):
    def __init__(self, pop_size: int = 100, **kwargs):
        super().__init__(
            pop_size=pop_size,
            sampling=LHS(),
            crossover=SBX(prob=0.9, eta=15),
            mutation=PM(eta=20),
            **kwargs,
        )
    # Override only what is new (e.g., _advance, survival, mating).
```

Ready-to-use components — prefer these over complementary implementations:

Building block	Import path	Avoids hand-coding
SBX crossover	<code>pymoo.operators.crossover.sbx.SBX</code>	Real-valued recombination
Polynomial mutation	<code>pymoo.operators.mutation.pm.PM</code>	Bounded mutation
Tournament selection	<code>pymoo.operators.selection.tournament.TournamentSelection</code>	Custom selection loops
Random / LHS sampling	<code>pymoo.operators.sampling.rnd.FloatRandomSampling, ...lhs.LHS</code>	Manual <code>np.random</code> init
Rank + crowding survival	<code>pymoo.operators.survival.rank_and_crowding.RankAndCrowding</code>	Custom NDS + crowding
Mating	<code>pymoo.core.mating.Mating</code>	Manual offspring generation
Non-dominated sorting	<code>pymoo.util.nds.non_dominated_sorting.NonDominatedSorting</code>	Custom Pareto ranking

Project-local, backend-aware variants live under `operators.*` (for example, `operators.crossover.sbx` and `operators.selection.tournament`) and are interchangeable; use them when you need JAX-aware or project-specific behavior. By delegating these to `pymoo/core`, a new algorithm typically reduces to one or two overridden methods.

13. Creating New Metrics

13.1. The Metric Contract

A metric is a *callable* that receives the obtained front and a context, returning a float:

```
def my_metric(front: np.ndarray, context: dict) -> float:
    # front: (N, M) -> N solutions, M objectives
    # context: 'problem', 'reference_front', 'n_obj', ...
    return float(computed_value)
```

13.2. Available Context

Key	Type	Description
problem	Problem	Instance of the problem being evaluated
reference_front	np.ndarray None	True Pareto front
reference_set	np.ndarray None	Reference set in the X space
n_obj	int	Number of objectives
pop_X	np.ndarray None	Decision variables of the population
pop_G	np.ndarray None	Constraint violations
feasible_mask	np.ndarray None	Feasibility mask

13.3. Example and Registration

```
# metrics/my_metric.py (or inside community_metrics.py)
import numpy as np
from typing import Any

def _metric_MyDist(front: np.ndarray, context: dict[str, Any]) -> float:
    pop_obj = np.atleast_2d(np.asarray(front, dtype=float))
    ideal = np.min(pop_obj, axis=0)
    distances = np.sqrt(np.sum((pop_obj - ideal) ** 2, axis=1))
    return float(np.mean(distances))

METRICS = {"MyDist": _metric_MyDist}
```

13.4. Reusing pymoo Indicators (Recommended)

Quality indicators (HV, IGD, IGD+, GD, GD+) are already implemented and validated in pymoo. A new metric should **wrap the relevant indicator** inside the METRICS contract rather than re-deriving the distance math — `community_metrics.py` does exactly this for hypervolume. The reference front and reference point arrive through the context, so the wrapper is only a couple of lines.


```
# Reuse pymoo indicators instead of recomputing distances
import numpy as np
from typing import Any
from pymoo.indicators.igd import IGD
from pymoo.indicators.hv import HV

def _metric_IGD(front: np.ndarray, context: dict[str, Any]) -> float:
    ref = context.get("reference_front")
    if ref is None:
        return float("nan")
    return float(IGD(np.asarray(ref, dtype=float))(np.asarray(front, dtype=float)))

def _metric_HV(front: np.ndarray, context: dict[str, Any]) -> float:
    ref = context.get("reference_front")
    if ref is None:
        return float("nan")
    ref_point = np.max(ref, axis=0) * 1.1
    return float(HV(ref_point=ref_point)(np.asarray(front, dtype=float)))

METRICS = {"IGD": _metric_IGD, "HV": _metric_HV}
```

Ready-to-use indicators — prefer these over complementary implementations:

Indicator	Import path	Requires
HV (Hypervolume)	pymoo.indicators.hv.HV	reference point
IGD	pymoo.indicators.igd.IGD	reference front
IGD+	pymoo.indicators.igd_plus.IGDPlus	reference front
GD	pymoo.indicators.gd.GD	reference front
GD+	pymoo.indicators.gd_plus.GDPlus	reference front

For non-dominated filtering of a raw population, reuse `pymoo.util.nds.non_dominated_sorting.NonDominatedSorting` rather than writing a custom dominance loop.

14. Creating New Operators

14.1. Categories

Category	pymoo base class	Directory
Crossover	pymoo.core.crossover.Crossover	operators/crossover/
Mutation	pymoo.core.mutation.Mutation	operators/mutation/
Selection	pymoo.core.selection.Selection	operators/selection/
Sampling	pymoo.core.sampling.Sampling	operators/sampling/
Repair	pymoo.core.repair.Repair	operators/repair/

14.2. Example: Crossover

```
# operators/crossover/my_crossover.py
import numpy as np
from pymoo.core.crossover import Crossover

class MyCrossover(Crossover):
    def __init__(self, alpha: float = 0.5, **kwargs):
        super().__init__(n_parents=2, n_offsprings=2, **kwargs)
        self.alpha = alpha

    def _do(self, problem, X, **kwargs):
        n_matings, _, n_var = X.shape
        Y = np.full((n_matings, self.n_offsprings, n_var), np.nan)
        for i in range(n_matings):
            p1, p2 = X[i, 0], X[i, 1]
            Y[i, 0] = self.alpha * p1 + (1 - self.alpha) * p2
            Y[i, 1] = (1 - self.alpha) * p1 + self.alpha * p2
        return Y
```

14.3. Example: Mutation

```
# operators/mutation/my_mutation.py
import numpy as np
from pymoo.core.mutation import Mutation

class MyMutation(Mutation):
    def __init__(self, sigma: float = 0.1, **kwargs):
        super().__init__(**kwargs)
        self.sigma = sigma

    def _do(self, problem, X, **kwargs):
        X = np.asarray(X, dtype=float)
        Y = X + np.random.normal(0, self.sigma, X.shape)
        return np.clip(Y, problem.xl, problem.xu)
```

15. Execution and Workflow

15.1. Launching the Application

```
python PymooLab.py
```

The application starts maximized, with a qt-material theme. All plugin directories are created automatically if absent, and the catalog (local + pymoo) is loaded.

15.2. Experimentation Flow

- **1. Select algorithms** — filter by type (single/multi/many) and encoding (real/binary/integer/permutation).
- **2. Select problems** — filter by family, objective type, and characteristics (constrained/dynamic).
- **3. Select metrics** — HV, IGD, GD, Spacing, DeltaP, etc.
- **4. Configure execution** — Max FE/generations, number of runs, seed mode (random/fixed/sequence), and backend (CPU/JAX).
- **5. Run** — the engine uses ProcessPoolExecutor to parallelize trials across isolated subprocesses, keeping the UI responsive.
- **6. Analyze** — convergence, Pareto fronts, statistical tests, and CSV/LaTeX export in the Results tab.
- **7. MCDM (optional)** — select the compromise solution from the obtained fronts.

15.3. Subprocess Worker

Each trial runs in a separate process via `pymoolab_process_worker.run_trial_in_process`, which imports `minimize` from `optimize` (an alias for `pymoo.optimize`). This isolation prevents a single algorithm's failures or memory leaks from affecting the main application.

16. Troubleshooting

Symptom	Likely cause	Action
Algorithm absent from the filters	Missing ALGORITHM_FLAGS or incompatible scope	Declare the correct flags in the algorithm module
"JAX available, but no GPU"	JAX installed without a CUDA/ROCm device	Use the CPU backend or install GPU-enabled jaxlib
Statistical test unavailable	SciPy missing or too few samples	Install scipy; ensure <code>>= min_samples</code> runs
LLM code rejected	Security/contract validation failed	Revise the generated code per the Ch. 11/13 contracts
Plugin not discovered	Missing <code>__init__.py</code> or required export	Ensure the class/factory is exported and <code>__init__.py</code> is present
Stale cache after editing a plugin	Outdated <code>__pycache__</code>	Run <code>python clean_pycache.py</code> and restart

Appendix A. Array Backend Acceleration (CPU / JAX / MLX)

PymooLab supports three interchangeable compute backends, resolved through a backend-descriptor registry (`core/registry/backends.py`). The `util/array_backend.py` module centralizes the active array module (`xp`), exposing NumPy by default, `jax.numpy` for the JAX backend, and `mlx.core` for the MLX backend on Apple Silicon. Accelerated artifacts follow the `_JAX` / `_MLX` suffix convention and are selected automatically by `backend_selection`, with a transparent fallback to the CPU implementation when a variant is absent.

Backend	Token	Array module	Variant suffix	Default dtype
CPU (NumPy)	<code>cpu</code>	<code>numpy</code>	<code>(none)</code>	<code>float64</code>
JAX (GPU/TPU)	<code>jax</code>	<code>jax.numpy</code>	<code>_JAX</code>	<code>float32</code>
MLX (Apple Silicon)	<code>mlx</code>	<code>mlx.core</code>	<code>_MLX</code>	<code>float32</code>

- **Default selection.** When MLX is detected (Apple Silicon with a working `mlx` runtime) it is selected by default; otherwise the default is CPU. JAX remains selectable on any platform.
- **Detection.** `detect_mlx_runtime()` and `detect_gpu_runtime()` probe MLX and JAX lazily (never importing them eagerly) and report availability and device.
- **Graceful fallback.** When the requested accelerator or a specific `_JAX/_MLX` variant is unavailable, execution degrades to the CPU automatically and results stay numerically equivalent within tolerance.
- The default accelerated dtype is `float32`, suitable for both GPU and Apple Silicon.

Appendix B. References

[1] Santos, T. F., de Souza, G., Carneiro, L. G. O. & Xavier, S. (2025). PymooLab: An Open-Source Visual Analytics Framework for Multi-Objective Optimization using LLM-Based Code Generation and MCDM. arXiv:2603.01345. DOI: 10.48550/arXiv.2603.01345.